

Introduction

Recently, there was polemic in the C++ community about the coroutines limitations, that they are stackless by design – they allocate the frame, that represents the coroutine snapshot in memory, on the **heap**, as the way to preserve its state at the suspension point. This way, it can restore the same state at the point of being resumed (`co_awaited`).

Basically, the memory must outlive the coroutine – cannot be local to the coroutine.

The sequence of creating the coroutine is:

- The memory needs to be available, either on the heap – or preallocated on the stack (Arena)
- The *promise_type* that stores the coroutine frame will be created in that memory: the memory can't be local to the promise type
- Eventually, the task that represents the coroutine will be created from promise type: the memory can't be local to the coroutine return type (Task) either

How can we override the default allocation strategy for coroutine frame?

Custom allocator to rescue

To be able to employ the **custom allocator** that works with coroutines – we need to override the global `::operator new/delete` in the `promise_type`: the respective overloads that allocate and release raw bytes. The compiler machinery will start the lifetime of the `promise_type` within given memory

```
template <bool lazy>
struct promise_type_base
{
    [[nodiscard]] static void* operator new(std::size_t size) noexcept
    {
        return ::utils::arena::DefaultFrameAllocator::allocate(size);
    }

    static void operator delete(void* ptr, std::size_t size) noexcept
    {
        ::utils::arena::DefaultFrameAllocator::deallocate(ptr, size);
    }
}
```

This is much more convenient than using what standard provides – the `promise_type::operator new()` overload

```
static void* operator new(std::size_t size, std::allocator_arg_t,
    MyAllocator& alloc)
{
    return alloc.allocate(size);
}
```

with tag dispatch `std::allocator_arg_t` that is used to overload the new operator, telling compiler to consider all arguments provided after.

This affects the Task signature – demanding adding the Allocator type for each coroutine definition, which is quite cumbersome

```
TaskLazy<void, MyAllocator>
my_coroutine(std::allocator_arg_t, MyAllocator& alloc, ...)
{
    // body
}
```

One way around, would be using type-erased, polymorphic allocator

```
// promise_type always uses pmr
static void* operator new(std::size_t size,
                          std::allocator_arg_t,
                          std::pmr::memory_resource* resource) {
    return resource->allocate(size, __STDCPP_DEFAULT_NEW_ALIGNMENT__);
}
```

so that Task signature remains intact

```
TaskLazy<void>
my_coroutine(std::allocator_arg_t, MyAllocator* alloc, ...)
{
    // body
}
```

with requirements on allocator – to implement `std::pmr::memory_resource` interface.

For overloading the operator `delete()` we can't use the same approach, with tag dispatch. The only way to work around this limitation is to stash the allocator reference into allocated header (along with payload – allocated raw bytes) and restore it at the point of deallocation.

There is a better way. We can use newly introduced *FixedSizePoolMemoryResource* as a global resource, that implements exactly the same – `std::pmr::memory_resource` interface. Intention is to utilize on the SBO as the primary allocation strategy – trying first to allocate coroutine frame on the stack, falling back to the heap allocation, in case that primary memory resource doesn't have enough free capacity.

It will be a simple wrapper, around the FBA pool - with small upgrades:

- It will be globally accessible – having internal static storage, and therefore
- It will be thread-safe

```
template <std::size_t BlockCount,
          std::size_t Alignment,
          std::size_t... BlockSizes>
class FrameAllocator final
{
    ...
}
```

```
private:
    using resource_type = FixedSizePoolMemoryResource<BlockCount,
                                                    Alignment,
                                                    BlockSizes...>;

    static inline resource_type resource_{}; // static storage
    static inline std::mutex mutex_{}; // thread-safe
```

The corresponding allocate/deallocate operations just delegate jobs to the internal allocator, with thread exclusive access

```
[[nodiscard]] static void* allocate(std::size_t size) noexcept
{
    try
    {
        std::lock_guard lock{mutex_};
        return resource_.allocate(size, Alignment);
    }
    catch (...)
    {
        return nullptr;
    }
}
```

Similar, for deallocation – we have

```
static void deallocate(void* ptr, std::size_t size) noexcept
{
    if (ptr == nullptr) [[unlikely]] return;

    std::lock_guard lock{mutex_};
    resource_.deallocate(ptr, size, Alignment);
}
```

Finally, we can provide default instantiation, for simpler usage on the client (Task) side – as singleton instance

```
using DefaultFrameAllocator = FrameAllocator<
    16, // BlockCount
    _STDCPP_DEFAULT_NEW_ALIGNMENT_, // Alignment
    64, // BlockSizes
    128,
    256,
    512 >;
```

Entire source code:

```
namespace utils::arena
{
    template <std::size_t BlockCount,
              std::size_t Alignment,
              std::size_t... BlockSizes>
    class FrameAllocator final
```

```

{
private:
    using resource_type = FixedSizePoolMemoryResource<BlockCount,
                                                Alignment, BlockSizes...>;

    static inline resource_type resource_{};
    static inline std::mutex mutex_{};

public:
    FrameAllocator() = default;

    // Non-copyable, non-movable (stateless handle to static resource)
    FrameAllocator(const FrameAllocator&) = delete;
    FrameAllocator& operator=(const FrameAllocator&) = delete;
    FrameAllocator(FrameAllocator&&) = delete;
    FrameAllocator& operator=(FrameAllocator&&) = delete;

    /**
     * Allocate raw bytes for a coroutine frame.
     * Delegates to FixedSizePoolMemoryResource: pool first,
     * heap fallback on exhaustion.
     *
     * @param size The frame size requested by the compiler
     * @return Pointer to allocated memory, or nullptr on failure
     */
    [[nodiscard]] static void* allocate(std::size_t size) noexcept
    {
        try
        {
            std::lock_guard lock{mutex_};
            return resource_.allocate(size, Alignment);
        }
        catch (...)
        {
            return nullptr;
        }
    }

    /**
     * Deallocate a coroutine frame.
     * Delegates to FixedSizePoolMemoryResource which uses owns()
     * internally to route to pool or fallback resource.
     *
     * @param ptr Pointer to the frame (as returned by allocate)
     * @param size The frame size (same value passed to allocate)
     */
    static void deallocate(void* ptr, std::size_t size) noexcept
    {
        if (ptr == nullptr) [[unlikely]] return;

        std::lock_guard lock{mutex_};
        resource_.deallocate(ptr, size, Alignment);
    }
};

/**
 * Default coroutine frame allocator configuration.
 */

```

```

* Block sizes chosen to cover typical coroutine frame sizes:
* - 64 bytes: minimal coroutines (no locals across suspend points)
* - 128 bytes: simple coroutines with a few locals
* - 256 bytes: moderate coroutines with captures/lambda's
* - 512 bytes: larger coroutines with multiple locals/awaits
*
* BlockCount of 16 allows up to 16 concurrent in-flight coroutines
* per size class.
* Adjust based on application's concurrency profile.
*/
using DefaultFrameAllocator = FrameAllocator<
    16, // BlockCount
    __STDCPP_DEFAULT_NEW_ALIGNMENT__, // Alignment
    64, // BlockSizes
    128,
    256,
    512 >;

} // namespace utils::arena

```

For the rest – read in my new book, it's coming soon...